

Build Task

One task: a working web app that takes audio — recorded live in the browser or uploaded as a file — runs it through an AI service, and turns what was said into a word cloud. Everything you need is in this document: the scope, the technical specification, what to submit and how you will be judged.

1 build

Record + upload

AI analysis

Word cloud

5 days

THE TASK	One web app — record or upload audio, analyse it with AI, render a word cloud
CONFIRM FIRST	Reply on the portal within 24 hours to say whether you are taking it on
TIME GIVEN	5 days from the date on the task mail
SUBMIT THROUGH	The hiring portal only
DELIVERABLES	Public GitHub repo, a live working URL, and a README in the repo
STACK	Entirely your choice — see Section 04

01 BEFORE YOU WRITE ANY CODE

Open the hiring portal and tell us whether you are taking this task on. Do it within 24 hours of the task mail landing. A yes or a no are both fine answers; silence is not, and it is the one response that ends the process on its own.

The five days run from the date on the task mail, not from the day you get around to confirming. Confirming late does not buy you extra time.

QUESTIONS

Ask them on the portal or by replying to the mail this came from. Asking a good question is not a mark against you. Guessing silently and getting it wrong is.

02 WHAT YOU ARE BUILDING, AND WHO FOR

We run live one-to-one mentorship sessions for school students. Mentors record those sessions, and afterwards nobody has time to listen to forty minutes of audio again. We want a tool that takes a recording and gives back a fast visual answer to one question: what was this session actually about?

That is the app. It has to do three things at once, and that is the hard part of this brief:

- A **mentor with no technical background** should be able to open it, record or upload, and get a result without anyone explaining the screen to them.
- The **result** should be readable in a glance — the words that dominated the session should be obvious, and the ones that barely came up should not compete with them.

- It should **behave sensibly when things go wrong**. A denied microphone, a 90 MB file, a silent recording, a failed API call. Any of these will happen, and the app should say something useful instead of freezing.

THE TONE WE ARE AIMING FOR

A tool, not a toy and not a dashboard. Clean, quick, obvious. Think "one screen that does one job well", not "landing page with a hero section and four feature cards".

03 THE BUILD

Four parts. All four are required, and they are one flow — not two separate apps bolted together. Recording and uploading are two doors into the same pipeline.

A • Record live audio in the browser

- Clear start and stop, with an unmistakable indication that recording is live.
- An elapsed timer while recording.
- Playback before the user commits to analysing it, and the option to discard and record again.
- Handle a denied or missing microphone with a message that tells the user what to do about it.

B • Upload an audio file

- File picker, and drag-and-drop if you want it. Accept the formats listed in Section 04.
- Reject anything else with a message that says *why* — not a silent failure and not a raw console error.
- Show the file name, size and duration once it is selected, and show upload progress.

C • Analyse it with an AI service

- Send the audio to an AI service that transcribes it and identifies the prominent terms.
- Strip filler and stopwords. Normalise sensibly — case, plurals, obvious variants of the same word.
- Counting raw word frequency with no AI in the loop is not what is being asked for, and we will be able to tell.

D • Render the word cloud

- Rendered on screen, with size reflecting prominence.
- Downloadable as a PNG.
- Existing word cloud libraries are allowed. Name them in your README.

The rules

- **Never leave the user staring at a frozen screen.** Transcription takes time. Every long operation needs a visible state — loading, progress, or an error that explains itself.
- **It has to run from your own README.** Clone, install, add a key, run. If we cannot get it working in ten minutes from your instructions, we cannot assess it.
- **The API key never goes in the repo.** Environment variable, with a `.env.example` committed in its place.
- **It should be usable on a phone.** Not a separate mobile design — just a layout that does not fall apart at 390 px.

TWO THINGS WE CHECK FIRST

A repo that will not run from its own README, and an API key sitting in the commit history. Both are basics, and both are checked before anything else is looked at.

04 TECHNICAL SPECIFICATION

Stack	Your choice. Any language, any framework, or none. Pick what you can move fastest in.
Backend	Your choice. A client-only app is acceptable as long as the AI step genuinely works and the key is not exposed.
Login / accounts	Not required. Building one counts against you — it is scope you were not asked for.
Persistence	Session-only is fine. Saving past analyses is a bonus, not a requirement.
Audio formats	MP3, WAV, M4A, AAC, OGG, WEBM, FLAC. Anything else is rejected with a clear message.
Limits	Up to 25 MB or 10 minutes, whichever is hit first. Enforce it, and say so in the UI before the user waits.
AI provider	Your choice, free tier is fine. We do not supply a key — use your own and keep it out of the repo.
Hosting	Any free tier — Vercel, Netlify, Render, Railway, Cloudflare. The live URL must actually work, AI step included.
Browsers	Current Chrome and Safari, desktop and mobile. Nothing older.
Repository	Public on GitHub, with real commits. One commit containing the whole project tells us nothing.

05 WHAT WE ARE NOT ASKING FOR

Skip these. Building them will not earn credit and will cost you the time you needed elsewhere:

- User accounts, sign-up, roles or an admin panel.
- Speaker separation — telling the mentor apart from the student.
- Live transcription streaming while the recording is still running.
- Multiple languages. English is enough.
- A mobile app, a design system, or a marketing landing page in front of the tool.

USE YOUR JUDGEMENT

We are not testing whether you can follow a feature list. We are testing whether you can take a vague real problem, decide what matters, build the part that matters properly, and explain why you cut the rest. Where this brief leaves a gap, make a decision and write it down in the README.

06 BONUS — OPTIONAL

Only worth attempting once the four required parts work end to end. A solid core beats a scattered pile of half-finished extras, and we would rather see the former.

- Show the transcript alongside the cloud, and let it be copied or downloaded.
- Let the user remove a word from the cloud and re-render without re-analysing.
- Save past analyses so a user can come back to one.
- Let the user pick a colour scheme or shape for the cloud.

07 WHAT TO SUBMIT

- **Public GitHub repository** — link posted on the portal.
- **Live deployed URL** — working, with the AI step functioning. Open it yourself in a private window before you submit.
- **README.md** in the repo. This is where your thinking gets marked, so it matters.

The README — short, plain, and it counts

No essays. Cover these seven things:

- What you built and what actually works. If something is half-done, say so — we mark honesty well above a hidden broken feature.
- How to run it locally: clone, install, environment variables, run. Exact commands, in order.
- Which AI service you used and why you chose that one.
- Two or three decisions you made and the reason behind each — a stack choice, a trade-off, something you deliberately did not build.
- Every library, component or template you used that is not your own code.
- Whether you used AI coding tools, and for what. Using them is allowed; not telling us is not.
- What you would do next if you had another week.

Before you submit: clone your own repo into a fresh folder, follow your own README line by line, and confirm it runs. A repo that only works on the machine it was written on is the most common way a good submission gets thrown out.

08 DEADLINE

Confirming	On the portal, within 24 hours of the task mail. Yes or no — both are fine, silence is not.
Time given	5 days from the date on the task mail. The exact closing time is shown on the portal, and the portal closes at that point.
Where	The hiring portal only. Links sent by email, WhatsApp or DM will not be considered.
Late work	Partial work submitted on time beats complete work submitted late.

Submitting early	Viewed positively and carried through the rest of the process — provided the work is finished, not rushed.
------------------	--

09 HOW YOUR WORK WILL BE ASSESSED

Core functionality	Do all four parts work, end to end, on the live URL — record, upload, analyse, render?	30%
Interface & experience	Can a non-technical mentor use it unaided? Layout, states, clarity, restraint, behaviour on a phone.	25%
Code quality	Structure, naming, readability, sane commits, no keys in the repo.	20%
The unhappy path	Denied microphone, oversized file, wrong format, silent audio, failed API call.	15%
Thinking	The README — did you make decisions and defend them, or just ship whatever came out?	10%

ON SCOPE

Volume of features is not a criterion. A small app where all four steps work reliably beats an ambitious one where the record button works and everything after it does not.

10 GROUND RULES

- **The work must be yours.** Libraries, starter templates and AI coding tools are all fine — declare them in the README.
- **Keep the repo public** until we tell you the review is finished.
- **This is an evaluation task, not client work.** It is unpaid, and we will not use your submission in any live product. If we later want to use any part of it, we will come back to you and agree terms first.
- **The brief is confidential.** The code you write is yours to keep in your portfolio once the process closes; this document is not to be shared or posted.
- **Questions are welcome.** Ask through the portal or by replying to the mail this came from.